JavaScript Code

```javascript
const API_BASE = (() => {
  if (window.location.hostname === "localhost" || window.location.hostname === "127.0.0.1") {
    if (window.location.port && window.location.port !== "8000") {
      return "http://localhost:8000";
    }
  }
  return window.location.origin;
})();

const elements = {
  navButtons: document.querySelectorAll(".pill-nav button"),
  heroButtons: document.querySelectorAll(".hero-actions button"),
  recommendationForm: document.getElementById("recommendation-form"),
  recommendationResults: document.getElementById("recommendation-results"),
  clusterCards: document.getElementById("cluster-cards"),
  benchmarkCollege: document.getElementById("benchmark-college"),
  benchmarkButton: document.getElementById("benchmark-button"),
  benchmarkResults: document.getElementById("benchmark-results"),
  recruiterChips: document.getElementById("recruiter-chips"),
  policyResults: document.getElementById("policy-results"),
  searchFocus: document.querySelector("details[data-field=focus]"),
  filterCollege: document.getElementById("filter-college"),
  filterDelivery: document.getElementById("filter-delivery"),
  searchButton: document.getElementById("search-button"),
  programTable: document.getElementById("program-table"),
  datasetMeta: document.getElementById("dataset-meta"),
  programCount: document.getElementById("program-count"),
  collegeCount: document.getElementById("college-count"),
  clusterCount: document.getElementById("cluster-count"),
  chatbotToggle: document.getElementById("chatbot-toggle"),
  chatbotPanel: document.getElementById("chatbot-panel"),
  chatbotClose: document.getElementById("chatbot-close"),
  chatbotReset: document.getElementById("chatbot-reset"),
  chatbotMessages: document.getElementById("chatbot-messages"),
  chatbotInput: document.getElementById("chatbot-input"),
  chatbotSend: document.getElementById("chatbot-send"),
  chatbotEmoji: document.getElementById("chatbot-emoji"),
  chatbotMic: document.getElementById("chatbot-mic"),
  emojiPanel: document.getElementById("emoji-panel"),
  floatingDashboard: document.getElementById("floating-dashboard"),
  floatingToggle: document.getElementById("floating-dashboard-toggle"),
  floatingBody: document.getElementById("floating-dashboard-body"),
  floatingPrograms: document.getElementById("float-programs"),
  floatingColleges: document.getElementById("float-colleges"),
  floatingClusters: document.getElementById("float-clusters"),
  floatingTopCategory: document.getElementById("float-top-category"),
  floatingTopCredential: document.getElementById("float-top-credential"),
  floatingTopRegion: document.getElementById("float-top-region"),
};
```

```javascript

const DEFAULT_GREETING =
  "Hi! I’m Olivia. Ask me about programs, colleges, recommendations, or skills.";

const profileQuestions = [
  { key: "name", prompt: "What is your name?" },
  { key: "age", prompt: "How old are you?" },
  { key: "education", prompt: "What is your education background?" },
  { key: "skills", prompt: "List your skills (comma-separated)." },
  { key: "interest", prompt: "What are your interests (comma-separated)?" },
];

const chatState = {
  started: false,
  step: 0,
  profile: {},
};

const QUESTION_STARTERS = ["what", "which", "who", "where", "when", "why", "how", "can", "do", "does", "is", "are"];

// Error handling concept: centralize logging + user-facing fallbacks.
// This keeps error messaging consistent and avoids duplicated catch blocks.
function reportUiError(context, error, options = {}) { // Logs the error and updates optional UI targets.
  // Provides a single place to enrich logs with context.
  // Safely handles missing targets or messages.
  const { target, message, mode = "text" } = options;
  console.error(`[${context}]`, error);
  if (!target || !message) return;
  if (mode === "html") {
    target.innerHTML = message;
  } else {
    target.textContent = message;
  }
}

function isQuestion(text) { // Checks whether a user input looks like a question.
  // Uses punctuation or common starters to infer intent.
  // Helps route questions to the smart reply handler.
  const trimmed = text.trim().toLowerCase();
  return (
    trimmed.endsWith("?") || QUESTION_STARTERS.some((starter) => trimmed.startsWith(starter))
  );
}

function isProfileIntent(text) { // Detects whether a user wants personalized recommendations.
  // Scans for recommendation-related keywords.
  // Drives the chat flow into profile collection.
  const input = text.toLowerCase();
  return (
```

```javascript
    input.includes("recommend") ||
    input.includes("suggest") ||
    input.includes("best college") ||
    input.includes("match")
  );
}

const chartInstances = {};
const chatCache = {
  colleges: [],
};

function scrollToSection(id) { // Smooth-scrolls to a section and updates active nav state.
  // Scrolls to the target section for navigation.
  // Toggles active styles for the selected nav item.
  document.getElementById(id).scrollIntoView({ behavior: "smooth" });
  elements.navButtons.forEach((button) => {
    button.classList.toggle("active", button.dataset.target === id);
  });
}

elements.navButtons.forEach((button) => {
  button.addEventListener("click", () => scrollToSection(button.dataset.target));
});

elements.heroButtons.forEach((button) => {
  button.addEventListener("click", () => scrollToSection(button.dataset.target));
});

if (elements.floatingToggle && elements.floatingDashboard && elements.floatingBody) {
  elements.floatingToggle.addEventListener("click", () => {
    const isCollapsed = elements.floatingDashboard.classList.toggle("is-collapsed");
    elements.floatingToggle.textContent = isCollapsed ? "+" : "–";
    elements.floatingToggle.setAttribute("aria-expanded", String(!isCollapsed));
  });
}

async function fetchJSON(url, options = {}) { // Fetches JSON with consistent error handling.
  // Throws on non-OK responses to surface failures.
  // Logs errors for easier debugging.
  try {
    const response = await fetch(url, options);
    if (!response.ok) {
      throw new Error(`Request failed: ${response.status}`);
    }
    return await response.json();
  } catch (error) {
    reportUiError("API request failed", error);
    throw error;
  }
}
```

```javascript

function createCard(title, subtitle, meta, link) { // Builds a reusable card markup for list sections.
  // Generates a consistent card layout for lists.
  // Safely handles missing subtitle, meta, or link.
  return `
    <article class="card">
      <h3>${title}</h3>
      <p>${subtitle || ""}</p>
      ${meta ? `<p class="meta">${meta}</p>` : ""}
      ${link ? `<a href="${link}" target="_blank">More details</a>` : ""}
    </article>
  `;
}

function formatDuration(item) { // Formats program duration values into a readable label.
  // Prefers year values, then month values, then program length.
  // Falls back to a not-listed label when missing.
  if (item.duration_years) {
    return `${item.duration_years} years`;
  }
  if (item.duration_months) {
    return `${item.duration_months} months`;
  }
  if (item.program_length) {
    return `${item.program_length} months`;
  }
  return "Not listed";
}

function formatFees(item) { // Formats tuition values into a readable label.
  // Joins domestic, international, and additional fee fields.
  // Returns a not-listed label when fees are absent.
  const domestic = item.tuition_domestic ? `Domestic: ${item.tuition_domestic}` : null;
  const international = item.tuition_international
    ? `International: ${item.tuition_international}`
    : null;
  const extra = item.tuition_additional ? `Additional: ${item.tuition_additional}` : null;
  return [domestic, international, extra].filter(Boolean).join(" • ") || "Not listed";
}

function renderRecommendations(recommendations) { // Renders recommendation cards or an empty-state message.
  // Builds cards for each recommendation result.
  // Displays a helpful empty state if none exist.
  if (!recommendations.length) {
    elements.recommendationResults.innerHTML = "<p>No matches found. Try adding more skills.</p>";
    return;
  }
```

```
  elements.recommendationResults.innerHTML = recommendations
    .map(
      (item) =>
        createCard(
          item.college_name,
          `Course: ${item.program_name}`,
          `Duration: ${formatDuration(item)} • Campus: ${item.campus_name || "Not listed"} • Fees: ${formatFees(
            item
          )}`,
          item.details_url || item.source_url
        )
    )
    .join("");
}

function renderClusters(clusters) { // Renders skill cluster cards and recruiter chips.
  // Converts cluster data into cards and chip lists.
  // Updates the displayed cluster count.
  const entries = Object.entries(clusters);
  elements.clusterCards.innerHTML = entries
    .map(([name, info]) =>
      createCard(
        name,
        `Programs: ${info.count}`,
        `Sample: ${info.sample_programs.filter(Boolean).slice(0, 3).join(", ")}`
      )
    )
    .join("");

  const listTag = entries.length > 2 ? "ol" : "ul";
  elements.recruiterChips.innerHTML = `
    <${listTag} class="chip-list">
      ${entries.map(([name]) => `<li class="chip">${name}</li>`).join("")}
    </${listTag}>
  `;
  elements.clusterCount.textContent = entries.length;
}

function renderBenchmark(result) { // Renders benchmarking summary for a selected college.
  // Handles empty results with a fallback message.
  // Shows total vs IT program counts and top categories.
  if (!result || !result.college) {
    elements.benchmarkResults.innerHTML = "<p>No benchmark data found.</p>";
    return;
  }
  elements.benchmarkResults.innerHTML = `
    <h3>${result.college}</h3>
    <p>Total Programs: <strong>${result.total_programs}</strong></p>
    <p>IT Programs: <strong>${result.it_programs}</strong></p>
    <p>Top IT Categories: ${result.top_categories
```

```
      .map(([name, count]) => `${name} (${count})`)
      .join(", ")}</p>
  `;
}

function renderPolicy(result) { // Renders policy analysis insights.
  // Builds a list of regional supply counts.
  // Appends a descriptive note for context.
  if (!result) return;
  const rows = result.supply_by_region
    .map(([region, count]) => `<li>${region}: ${count}</li>`)
    .join("");
  const listTag = result.supply_by_region.length > 2 ? "ol" : "ul";
  elements.policyResults.innerHTML = `
    <h3>IT Program Supply by Region</h3>
    <${listTag}>${rows}</${listTag}>
    <p class="meta">${result.note}</p>
  `;
}

function renderPrograms(items) { // Renders program rows for the data explorer table.
  // Builds rows with core program fields.
  // Shows a single empty-state row when needed.
  if (!items.length) {
    elements.programTable.innerHTML = `<tr><td colspan="5">No programs found.</td></tr>`;
    return;
  }
  elements.programTable.innerHTML = items
    .map(
      (item) => `
      <tr>
        <td>${item.college_name}</td>
        <td>${item.program_name}</td>
        <td>${item.credential_type || ""}</td>
        <td>${item.delivery_format || ""}</td>
        <td>${item.program_category || ""}</td>
      </tr>
    `
    )
    .join("");
}

function getColorPalette(count) { // Generates a repeating color palette for charts.
  // Cycles through a preset palette for consistency.
  // Returns a color list sized to the data set.
  const palette = [
    "#38bdf8",
    "#f97316",
    "#a855f7",
    "#22c55e",
    "#eab308",
    "#ef4444",
```

```javascript
    "#14b8a6",
    "#6366f1",
    "#f472b6",
    "#84cc16",
  ];
  const colors = [];
  for (let i = 0; i < count; i += 1) {
    colors.push(palette[i % palette.length]);
  }
  return colors;
}

function getTopLabel(entries) { // Returns the top label from a stats list.
  // Safely handles missing or empty entries.
  if (!Array.isArray(entries) || entries.length === 0) {
    return "Not listed";
  }
  return entries[0][0];
}

function renderChart(elementId, label, data) { // Creates or updates a chart for a dataset.
  // Destroys existing charts to avoid duplicates.
  // Creates either a doughnut or bar chart by ID.
  const [labels, values] = [data.map((item) => item[0]), data.map((item) => item[1])];
  if (chartInstances[elementId]) {
    chartInstances[elementId].destroy();
  }
  const ctx = document.getElementById(elementId).getContext("2d");
  const colors = getColorPalette(values.length);
  chartInstances[elementId] = new Chart(ctx, {
    type: elementId === "delivery-chart" ? "doughnut" : "bar",
    data: {
      labels,
      datasets: [
        {
          label,
          data: values,
          backgroundColor: colors,
          borderColor: "rgba(15, 23, 42, 0.6)",
          borderWidth: 1,
        },
      ],
    },
    options: {
      responsive: true,
      plugins: {
        legend: { display: elementId === "delivery-chart" },
      },
      scales: elementId === "delivery-chart" ? {} : {
        x: { ticks: { color: "#e5e7eb" } },
        y: { ticks: { color: "#e5e7eb" }, grid: { color: "rgba(148, 163, 184, 0.2)" } },
```

```javascript
      },
    },
  });
}

async function loadInitialData() { // Loads dashboard data, updates charts, and caches lists.
  // Fetches core datasets in parallel for speed.
  // Updates UI counters, charts, and selectors.
  try {
    const [colleges, clusters, stats, programs] = await Promise.all([
      fetchJSON(`${API_BASE}/api/colleges`),
      fetchJSON(`${API_BASE}/api/skill-clusters`),
      fetchJSON(`${API_BASE}/api/stats`),
      fetchJSON(`${API_BASE}/api/programs?limit=15`),
    ]);

    chatCache.colleges = colleges;

    elements.filterCollege.innerHTML =
      `<option value="">All Colleges</option>` +
      colleges.map((college) => `<option value="${college}">${college}</option>`).join("");
    elements.benchmarkCollege.innerHTML = colleges
      .map((college) => `<option value="${college}">${college}</option>`)
      .join("");

    renderClusters(clusters.clusters);
    renderPrograms(programs.items);

    renderChart("category-chart", "Programs by Category", stats.categories);
    renderChart("credential-chart", "Programs by Credential", stats.credentials);
    renderChart("delivery-chart", "Programs by Delivery", stats.delivery);
    renderChart("region-chart", "Programs by Region", stats.regions);

    elements.programCount.textContent = programs.total;
    elements.collegeCount.textContent = colleges.length;
    elements.datasetMeta.textContent = `Loaded ${programs.total} IT program records across ${colleges.length} colleges.`;

    if (elements.floatingPrograms) {
      elements.floatingPrograms.textContent = programs.total;
      elements.floatingColleges.textContent = colleges.length;
      elements.floatingClusters.textContent = Object.keys(clusters.clusters || {}).length;
      elements.floatingTopCategory.textContent = getTopLabel(stats.categories);
      elements.floatingTopCredential.textContent = getTopLabel(stats.credentials);
      elements.floatingTopRegion.textContent = getTopLabel(stats.regions);
    }
  } catch (error) {
    reportUiError("Initial data load", error, {
      target: elements.datasetMeta,
      message: "Unable to load API data. Make sure the backend is running.",
```

```javascript
    });
  }
}

async function loadPolicy() { // Loads policy analysis data and renders the panel.
  // Fetches the policy endpoint and updates the panel.
  // Shows a friendly error message on failure.
  try {
    const result = await fetchJSON(`${API_BASE}/api/policy`);
    renderPolicy(result);
  } catch (error) {
    reportUiError("Policy load", error, {
      target: elements.policyResults,
      message: "<p>Unable to load policy data.</p>",
      mode: "html",
    });
  }
}

function getCheckedValues(container) { // Collects checked values from a details checkbox list.
  // Reads checked inputs and returns their values.
  // Guards against missing containers.
  if (!container) return [];
  return Array.from(container.querySelectorAll("input[type=checkbox]:checked")).map(
    (input) => input.value
  );
}

function updateSummaryText(detailsEl, fallback) { // Updates the summary text of a multi-select details element.
  // Replaces summary text with selected values.
  // Falls back to a default label when empty.
  if (!detailsEl) return;
  const summary = detailsEl.querySelector("summary");
  const values = getCheckedValues(detailsEl);
  summary.textContent = values.length ? values.join(", ") : fallback;
}

document.querySelectorAll(".multi-select").forEach((detailsEl) => {
  const fallback = detailsEl.querySelector("summary")?.textContent || "Select options";
  detailsEl.addEventListener("change", () => updateSummaryText(detailsEl, fallback));
});

elements.recommendationForm.addEventListener("submit", async (event) => {
  event.preventDefault();
  try {
    const formData = new FormData(event.target);
    const selectedSkills = getCheckedValues(
      event.target.querySelector("details[data-field=skills]")
    );
    const selectedGoals = getCheckedValues(
```

```javascript
      event.target.querySelector("details[data-field=goals]")
    );
    const payload = {
      education: formData.get("education") || "",
      skills: selectedSkills,
      goals: selectedGoals,
      delivery_preference: formData.get("delivery") || null,
      region_preference: formData.get("region") || null,
      max_duration_years: formData.get("duration")
        ? Number(formData.get("duration"))
        : null,
    };

    const result = await fetchJSON(`${API_BASE}/api/recommendations`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(payload),
    });

    renderRecommendations(result.recommendations || []);
  } catch (error) {
    reportUiError("Recommendations", error, {
      target: elements.recommendationResults,
      message: "<p>Unable to load recommendations. Please try again.</p>",
      mode: "html",
    });
  }
});

elements.searchButton.addEventListener("click", async () => {
  try {
    const selectedFocus = getCheckedValues(elements.searchFocus)
      .filter(Boolean)
      .join(" ");
    const params = new URLSearchParams({
      query: selectedFocus || "",
      college: elements.filterCollege.value || "",
      delivery: elements.filterDelivery.value || "",
      limit: 25,
    });

    const result = await fetchJSON(`${API_BASE}/api/programs?${params.toString()}`);
    renderPrograms(result.items || []);
  } catch (error) {
    reportUiError("Program search", error, {
      target: elements.programTable,
      message: "<tr><td colspan=\"5\">Unable to load programs. Please try again.</td></tr>",
      mode: "html",
    });
  }
});

```

```javascript
elements.benchmarkButton.addEventListener("click", async () => {
  const college = elements.benchmarkCollege.value;
  if (!college) return;
  try {
    const result = await fetchJSON(
      `${API_BASE}/api/benchmark?college=${encodeURIComponent(college)}`
    );
    renderBenchmark(result);
  } catch (error) {
    reportUiError("Benchmark load", error, {
      target: elements.benchmarkResults,
      message: "<p>Unable to load benchmark data.</p>",
      mode: "html",
    });
  }
});

loadInitialData();

loadPolicy();

function appendChatMessage(text, role = "bot") { // Appends a chat bubble to the chatbot transcript.
  // Adds a message element to the chat container.
  // Scrolls to keep the latest message in view.
  if (!elements.chatbotMessages) return;
  const message = document.createElement("div");
  message.className = `message ${role}`;
  message.textContent = text;
  elements.chatbotMessages.appendChild(message);
  elements.chatbotMessages.scrollTop = elements.chatbotMessages.scrollHeight;
}

function resetChat() { // Resets chatbot state and shows the default greeting.
  // Clears the message list and resets state.
  // Restarts the greeting for a fresh session.
  if (!elements.chatbotMessages) return;
  elements.chatbotMessages.innerHTML = "";
  appendChatMessage(DEFAULT_GREETING, "bot");
  chatState.started = false;
  chatState.step = 0;
  chatState.profile = {};
}

function getOliviaReply(question) { // Returns a simple rule-based response for common queries.
  // Uses keyword checks for fast canned responses.
  // Provides guidance to feature sections.
  const input = question.toLowerCase();

  if (input.includes("recommend")) {
    return "Use the Student Recommendations section. Choose education, skills, goals, and I will rank matching IT programs.";
```

```
  }
  if (input.includes("skill") || input.includes("cluster")) {
    return "Open Academic Advisor View to see skill clusters like Cybersecurity, Data & AI, and Software Development.";
  }
  if (input.includes("benchmark")) {
    return "In College Benchmarking, pick a college to compare total programs vs IT programs and top categories.";
  }
  if (input.includes("policy") || input.includes("region")) {
    return "Policy Analysis shows IT program supply by region to help planning and capacity decisions.";
  }
  if (input.includes("dashboard") || input.includes("chart")) {
    return "The Dashboard visualizes categories, credentials, and delivery formats using live dataset counts.";
  }
  if (input.includes("data") || input.includes("dataset") || input.includes("explorer")) {
    return "Use Data Explorer to filter programs by keyword, college, and delivery format.";
  }
  if (input.includes("ai")) {
    return "This project uses AI/NLP to classify programs and recommend the best fits for student profiles.";
  }
  if (input.includes("hello") || input.includes("hi")) {
    return "Hello! I'm Olivia. Ask me about recommendations, skill clusters, or program comparisons.";
  }

  return "I can help with recommendations, skill clusters, benchmarking, policy analysis, and data explorer. Try asking about one of these.";
}

async function getOliviaSmartReply(question) { // Returns a smarter reply based on live search results.
  // Pulls search results to craft a richer reply.
  // Falls back to rule-based replies on errors.
  try {
    const result = await fetchJSON(
      `${API_BASE}/api/search?query=${encodeURIComponent(question)}&limit=6`
    );

    const parts = [];

    if (result.colleges && result.colleges.length) {
      const college = result.colleges[0];
      const topCategories = (college.top_categories || [])
        .map(([name, count]) => `${name} (${count})`)
        .join(", ");
      parts.push(
```

```javascript
        `${college.college}: ${college.it_programs} IT programs out of ${college.total_programs}.` +
          (topCategories ? ` Top categories: ${topCategories}.` : "")
      );
    }

    if (result.programs && result.programs.length) {
      const names = result.programs
        .slice(0, 6)
        .map((item) => `${item.program_name} (${item.college_name})`)
        .filter(Boolean);
      if (names.length) {
        parts.push(`Suggested programs: ${names.join("; ")}.`);
      }
    }

    if (parts.length) {
      return parts.join(" ");
    }

    return getOliviaReply(question);
  } catch (error) {
    reportUiError("Smart reply", error);
    return "I'm having trouble accessing live data. Try again in a moment.";
  }
}

async function handleChatSend() { // Handles chatbot form input and conversation flow.
  // Manages profile collection and Q&A flow.
  // Sends recommendations when profile is complete.
  const text = elements.chatbotInput.value.trim();
  if (!text) return;
  appendChatMessage(text, "user");
  elements.chatbotInput.value = "";
  try {
    if (!chatState.started) {
      if (isProfileIntent(text)) {
        chatState.started = true;
        chatState.step = 0;
        appendChatMessage(profileQuestions[0].prompt, "bot");
        return;
      }

      if (isQuestion(text)) {
        const reply = await getOliviaSmartReply(text);
        appendChatMessage(reply, "bot");
        return;
      }

      appendChatMessage(
        "Ask me a question or say 'recommend' to start a personalized college match.",
        "bot"
      );
```

```javascript
        return;
      }

      if (chatState.step < profileQuestions.length) {
        const { key } = profileQuestions[chatState.step];
        chatState.profile[key] = text;
        chatState.step += 1;

        if (chatState.step < profileQuestions.length) {
          appendChatMessage(profileQuestions[chatState.step].prompt, "bot");
          return;
        }

        const skills = (chatState.profile.skills || "")
          .split(",")
          .map((value) => value.trim())
          .filter(Boolean);
        const goals = (chatState.profile.interest || "")
          .split(",")
          .map((value) => value.trim())
          .filter(Boolean);

        const result = await fetchJSON(`${API_BASE}/api/recommendations`, {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({
            education: chatState.profile.education || "",
            skills,
            goals,
            limit: 8,
          }),
        });

        const recommendations = result.recommendations || [];
        const seen = new Set();
        const ordered = [];
        recommendations.forEach((item) => {
          const name = item.college_name || "Unknown College";
          if (!seen.has(name)) {
            seen.add(name);
            ordered.push({
              college: name,
              program: item.program_name || "Program",
            });
          }
        });

        if (!ordered.length) {
          appendChatMessage(
            "Thanks! I couldn't find matches yet. Try different skills or interests.",
            "bot"
          );
          return;
```

```javascript
      }

      const list = ordered
        .slice(0, 6)
        .map((item, index) => `${index + 1}) ${item.college} — ${item.program}`)
        .join("\n");

      appendChatMessage(
        `Thanks ${chatState.profile.name || "there"}! Here are the best colleges in order:\n${list}`,
        "bot"
      );
      return;
    }

    const reply = await getOliviaSmartReply(text);
    appendChatMessage(reply, "bot");
  } catch (error) {
    reportUiError("Chat send", error);
    appendChatMessage(
      "I had trouble reaching the data service. Please try again in a moment.",
      "bot"
    );
  }
}

const on = (element, event, handler) => { // Safely wires event listeners only when elements exist.
  // Avoids attaching listeners when elements are missing.
  // Keeps event wiring concise and consistent.
  if (element) {
    element.addEventListener(event, handler);
  }
};

on(elements.chatbotToggle, "click", () => {
  if (!elements.chatbotPanel || !elements.chatbotToggle) return;
  const isOpen = elements.chatbotPanel.classList.toggle("open");
  elements.chatbotPanel.setAttribute("aria-hidden", String(!isOpen));
  elements.chatbotToggle.setAttribute("aria-expanded", String(isOpen));
});

on(elements.chatbotClose, "click", () => {
  if (!elements.chatbotPanel || !elements.chatbotToggle) return;
  elements.chatbotPanel.classList.remove("open");
  elements.chatbotPanel.setAttribute("aria-hidden", "true");
  elements.chatbotToggle.setAttribute("aria-expanded", "false");
});

on(elements.chatbotReset, "click", resetChat);
on(elements.chatbotSend, "click", handleChatSend);
on(elements.chatbotInput, "keydown", (event) => {
  if (event.key === "Enter") {
```

```javascript
    event.preventDefault();
    handleChatSend();
  }
});

on(elements.chatbotEmoji, "click", () => {
  if (!elements.emojiPanel) return;
  const isOpen = elements.emojiPanel.classList.toggle("open");
  elements.emojiPanel.setAttribute("aria-hidden", String(!isOpen));
});

if (elements.emojiPanel) {
  elements.emojiPanel.querySelectorAll("button").forEach((button) => {
    button.addEventListener("click", () => {
      if (!elements.chatbotInput) return;
      elements.chatbotInput.value += button.textContent;
      elements.chatbotInput.focus();
    });
  });
}

let recognition = null;
if ("webkitSpeechRecognition" in window || "SpeechRecognition" in window) {
  const SpeechRecognition = window.SpeechRecognition || window.webkitSpeechRecognition;
  recognition = new SpeechRecognition();
  recognition.lang = "en-US";
  recognition.interimResults = false;
  recognition.maxAlternatives = 1;
}

on(elements.chatbotMic, "click", () => {
  try {
    if (!recognition) {
      appendChatMessage("Voice input is not supported in this browser.", "bot");
      return;
    }
    recognition.start();
  } catch (error) {
    reportUiError("Voice input", error);
    appendChatMessage("Voice input failed. Please try again.", "bot");
  }
});

if (recognition) {
  recognition.addEventListener("result", (event) => {
    const transcript = event.results[0][0].transcript || "";
    if (elements.chatbotInput) {
      elements.chatbotInput.value = transcript;
      elements.chatbotInput.focus();
    }
  });

```

```
  recognition.addEventListener("error", () => {
    appendChatMessage("Voice input failed. Please try again.", "bot");
  });
}

resetChat();

```